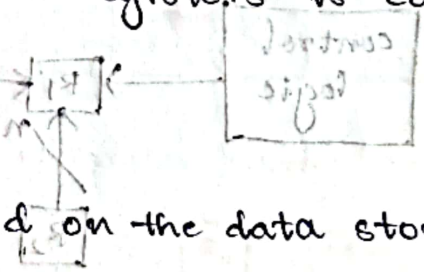
2. Register Transfer Language & Micro-Operation

→ Register transfer language (RTL)
→ Register transfer
→ Bus transfer
→ Memory transfer.
→ Arithmatic micro-operations.
→ Logical micro-operations.
→ Shift micro-operations
→ Arithmetic-logic-shift unit.

Basics of register & RTL:

Register: It is a fast, small temporary storage devic inside the CPU.

→ To execute the instruction, the data must be transfered from one register to another register and this data transfer blw the registers is called register transfer.



Micro-operations:

The operations executed on the data stored in the registers are called micro-operations.

Register transfer language (RTL):

RTL is a symbolic way of representing describing data transfer blw the registers and micro-operation on the registers.

Register transfer:

Inside the CPU, the data keeps moving from one register to another register. This is called register transfer.

Scanned with OKEN Scanner

Example for RTL description:

$R_1 \leftarrow R_2$ (destination ← source)

$R_1 = 10$, $R_2 = 8$

$R_1 \leftarrow R_2 \Rightarrow R_1 = 8, R_2 = 8$

meaning:

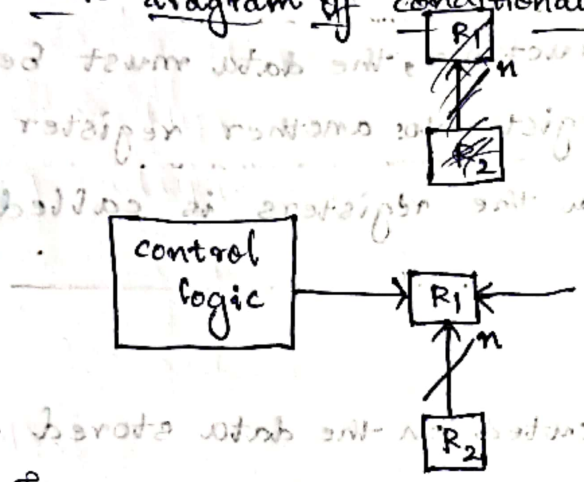
- The content of R_2 is transfered to R_1
- The content of R_2 is unchanged.
- The new content is loaded in R_1
- ← → replacement symbol.

→ We can also include control functions in the of register transfer.

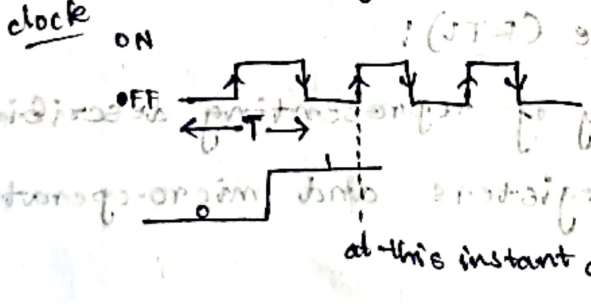
Ex: If $(P_1 = 1)$ then $R_1 \leftarrow R_2$

RTL description:- $P_1 : R_1 \leftarrow R_2$

Block diagram of conditional data transfer:



Timing diagram:



- To occur data transfer the required conditions ar

(1) The output lines of source must be connectioned to input lines of destination.

(2) Transfer occured on clock edge (rising or falling)

(3) while the clock edge occur, the control function must be enabled.

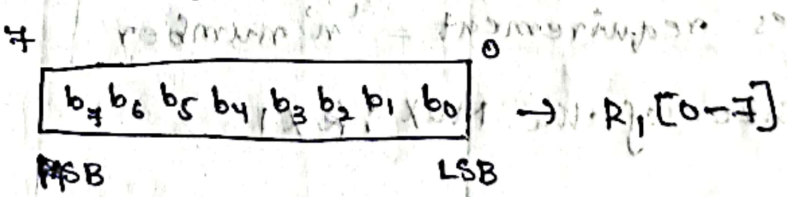
Basic Symbols used in RTL:

Symbol	Meaning	Examples
Capital letters	To name registers	MAR ← Memory Address Register PC ← program counter IR ← Instructor register R_1, R_2 → General
( )	Part of the register	PC (L) PC (0-7)
:	conditional data transfer	P: $R_1 \leftarrow R_2$
,	Simultaneous Operations	$R_1 \leftarrow R_2$, $R_3 \leftarrow R_4$
←	Transfer of data	$R_1 \leftarrow R_2$

Note:

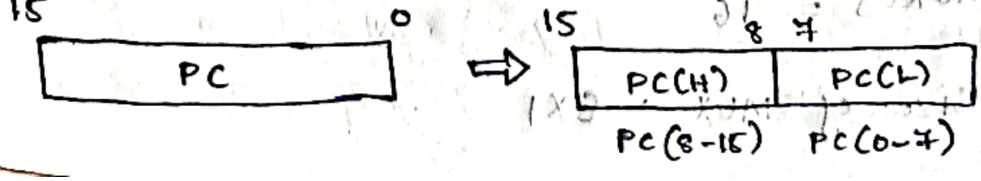
→ The registers are represented using rectangular box and write its name in it. R1 MAR

→ The binary values / contents are numbered always from ~~left to~~ LSB to RSB.



→ The register can be divided into two parts

ex: 16-bit PC can be divided into two parts as



Scanned with OKEN Scanner

Bus Transfer:

Bus: A bus is a set of common lines which is used to transfer binary information b/w registers.

* Only one register at a time can place the data on bus.

Bus structure

* The common bus structure / system can be constructed by two ways

  (1) Bus transfer by using multiplexers

  (2) Bus transfer by using tri-state-buffers.

(1) Bus transfer by using multiplexers:

→ The bus consists of multiple lines and each line uses one multiplexer.

→ Each multiplexer selects one-bit from the register among multiple registers and the selected register is connected to the bus.

General rule for selection of MUXes:

→ For k = no. of registers

Each register consists of 'n'-bits.

MUXes requirement = 'n' number

size of the MUX : k x 1

Ex: 8 - no. of registers

Each register consists of 16-bits.

No. of MUXes = 16

size of MUX = 8x1

Bus using gates for 4-bit registers

To

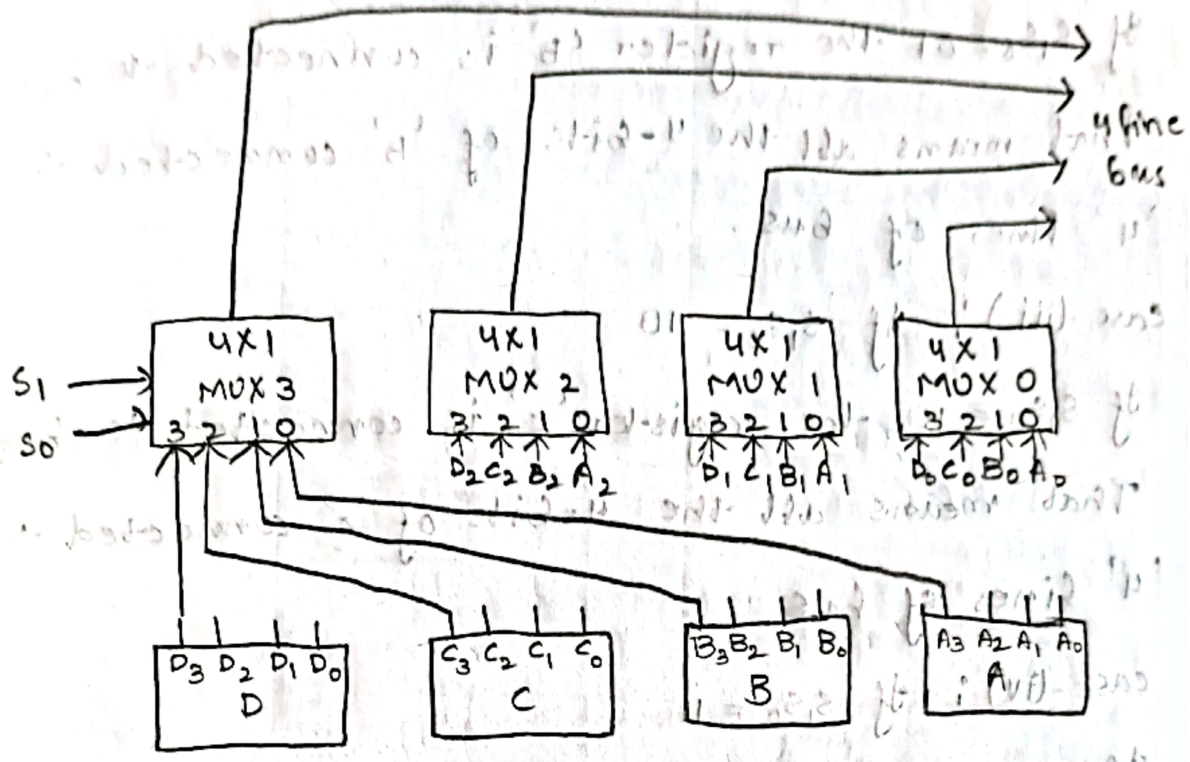


Fig. Bus System for 4-registers.

The common bus system for 4-registers as shown in figure above in which each register consists of 4-bits.

→ The work flow of the bus system can be understanding by considering 4 cases.

case-1: If $S_1S_0 = 00$

If $S_1S_0 = 00$, the register 'A' connected to Bus

That means all the 4-bits of 'A', connected to '4' lines of bus.

case-(ii) : If $S_1S_0 = 01$

If $S_1S_0 = 01$ the register 'B' is connected to bus.

That means all the 4-bits of 'B' connected to '4' lines of bus.

case-(iii) : If $S_1S_0 = 10$

If $S_1S_0 = 10$ the register 'C' is connected to bus.

That means all the 4-bits of 'C' connected to '4' lines of bus.

case-(iv) : If $S_1S_0 = 11$

If $S_1S_0 = 11$ the register 'D' is connected to bus.

That means all the 4-bits of 'D' connected to '4' lines of bus.

* No. of registers - 4,

So, each register required one 4x1 MUX

Total MUX : 4

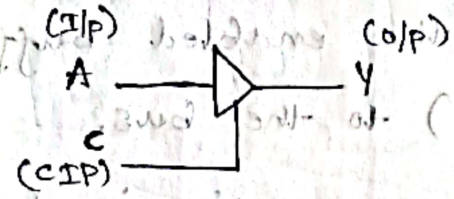
write beside diagram

* The selection lines S_1 S_0 are common to all MUXes.

(2) Bus-transfer using tri-state buffer:

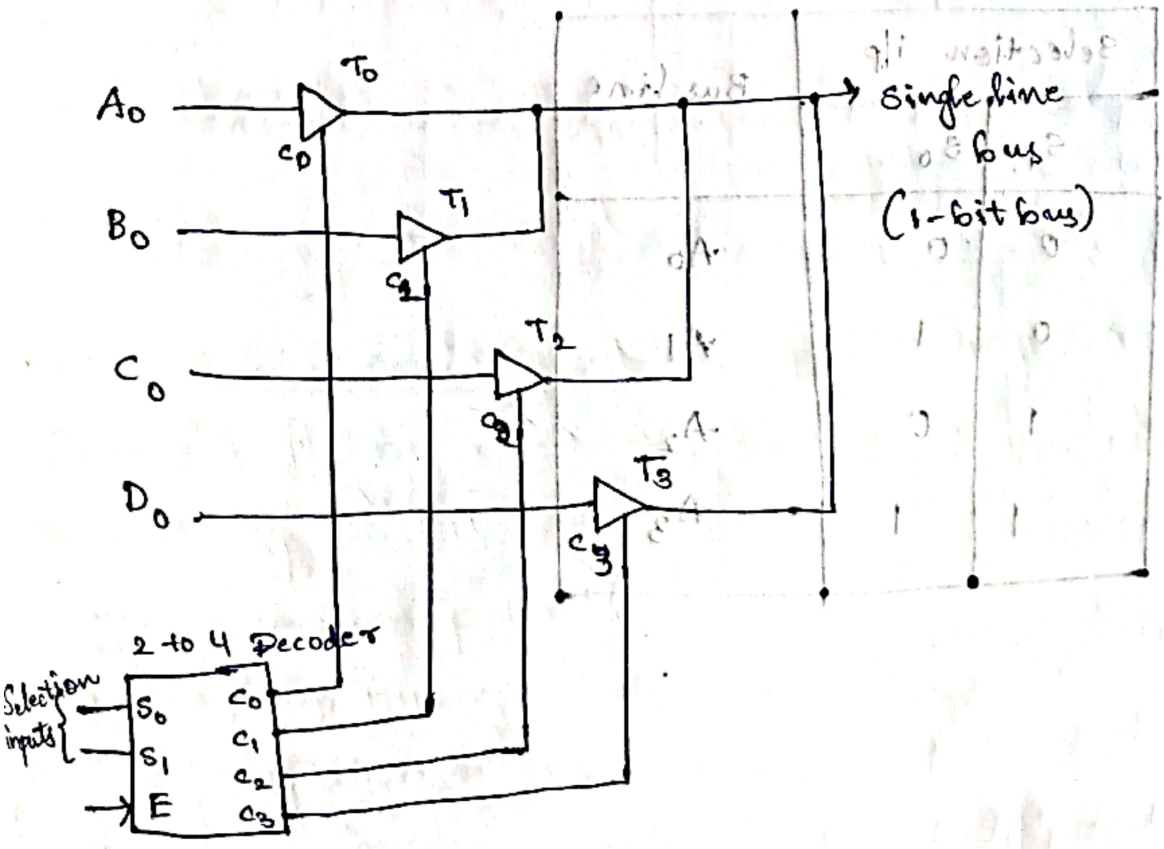
The bus structure for data transfer among the registers can be implemented by using tri-state buffer.

→ Tri-state-buffer is a 3 terminal device; one terminal for input and one terminal for output and third terminal called control input as shown in below figure.



* If C=1, then Y ← A.
* If C=0, (then the o/p disconnects from the i/p.)
(High Impedance state)

4-bit Bus structure:



Scanned with OKEN Scanner

A one-bit bus structure using

as shown in figure above.

→ In this structure, the register selection can be done by using tri-state-buffer with the help of 2--to-4 decoder.

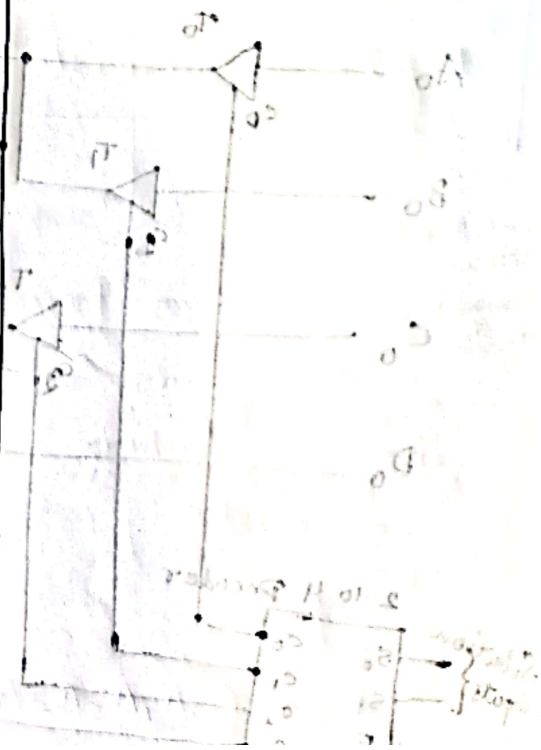
→ The decoder activates exactly one of its outputs according to their selection inputs (S_1, S_0).

→ The active decoder output enables the corresponding 3-state-buffer and the enabled buffer passes its input ($A_0/B_0/C_0/D_0$) to the bus.

→ The remaining buffers remain in high impedance state. Therefore, only one register bit appears on the bus line according to our selection of S_1 & S_0 as shown in below function table.

Function Table:

Selection i/p		Bus line
S_1	S_0	
0	0	A_0
0	1	A_1
1	0	A_2
1	1	A_3



Scanned with OKEN Scanner

Memory Transfer :

→ The memory transfer involves two operations :

(1) Read operation (2) Write operation.

(1) Read Operation :

→ It is used to retrive the data from the specified memory location. Memory address stored in AR and the data is transferred to R_1

RTL notation : READ : $R_1 \leftarrow M[AR]$

Meaning -

→ Read the data at address AR

→ Load that data into register R_1

(2) Write Operation :

→ It is used to store the data in a prescribed location of memory.

RTL notation : WRITE : $M[AR] \leftarrow R_1$

Meaning -

→ The content of R_1 stored in the memory location of AR.

Micro-Operations :

→ The operations executed on the data stored in the registers are called micro-operations.

→ Types of micro-operations :

1. Register transfer micro-operations.
2. Arthimetic micro-operations.
3. Logic micro-operations.
4. Shift micro-operations.

Scanned with OKEN Scanner

Arithmetic micro-operations:

→ The arithmetic operations executed on the numerical data stored in the registers.

→ The general arithmetic micro-operations are listed in the below table

Symbol Representation	Description
$R_3 \leftarrow R_1 + R_2$	Add R_1 & R_2 & result loaded in R_3
$R_3 \leftarrow R_1 - R_2$	Subtract R_1 & R_2, result loaded in R_3
$R_1 \leftarrow \overline{R_1}$	Complement R_1 and again loaded in R_1 ($1^s c$)
$R_2 \leftarrow \overline{R_1} + 1$	($2^s c$)
$R_3 \leftarrow R_1 + \overline{R_2} + 1$	$2^s c$ Subtraction
$R_1 \leftarrow R_1 + 1$	R_1 incremented by 1
$R_1 \leftarrow R_1 - 1$	R_1 decremented by 1

* The most common arithmetic micro-operations are addition, subtraction, increment and decrement.

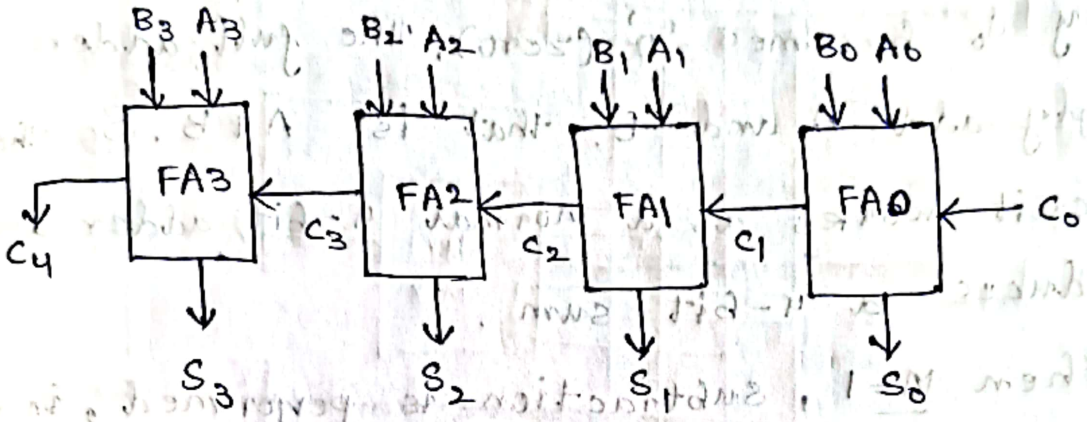
Binary Adder:

→ The 4-bit ripple carry binary adder as shown in figure below, it can add two 4-bit binary number $A(A_3A_2A_1A_0)$ and $B(B_3B_2B_1B_0)$

→ four full adders are connected in cascading manner to conclude 4-bit addition and at each stage of full adder, add bits of A and B along with incoming carry and produces sum and carry out which is propogated to next stage

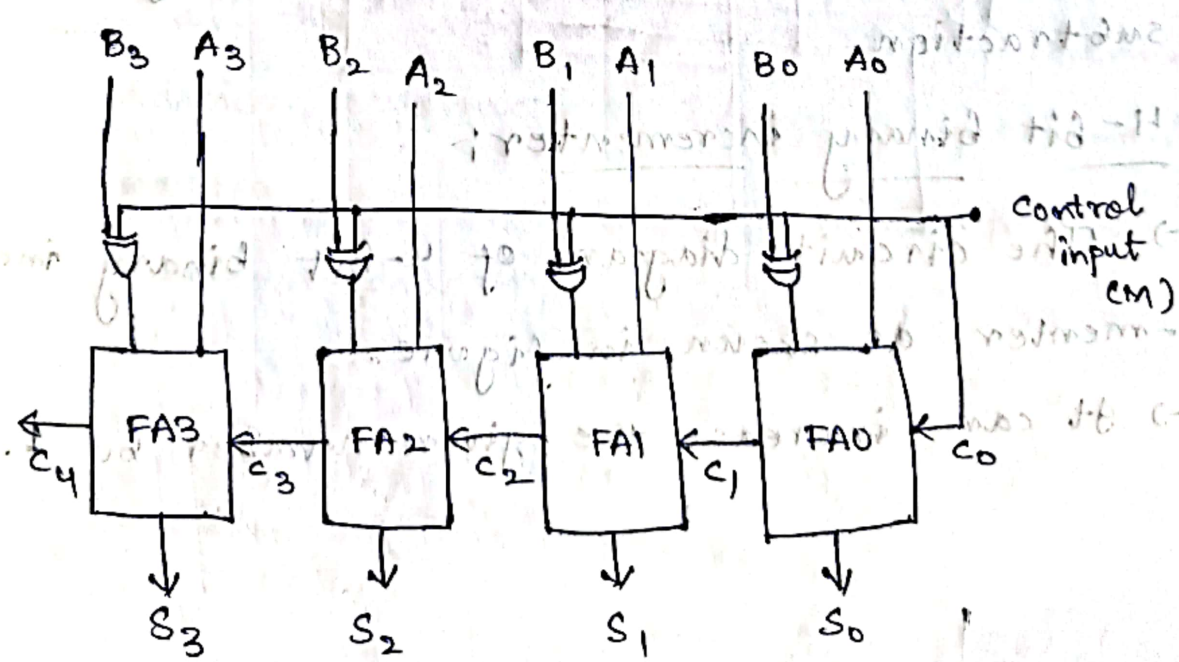
Ripple carry adder: $A \rightarrow A_3 A_2 A_1 A_0$

$B \rightarrow B_3 B_2 B_1 B_0$



4- bit adder - subtractor circuit :

→ The 4-bit adder - subtractor circuit as shown in figure below. It can perform either addition or subtraction using the same hardware.



Scanned with OKEN Scanner

Operations:

→ There are two i/p's A and B and one control signal (M) and the value of M decides the operation either addition or subtraction.

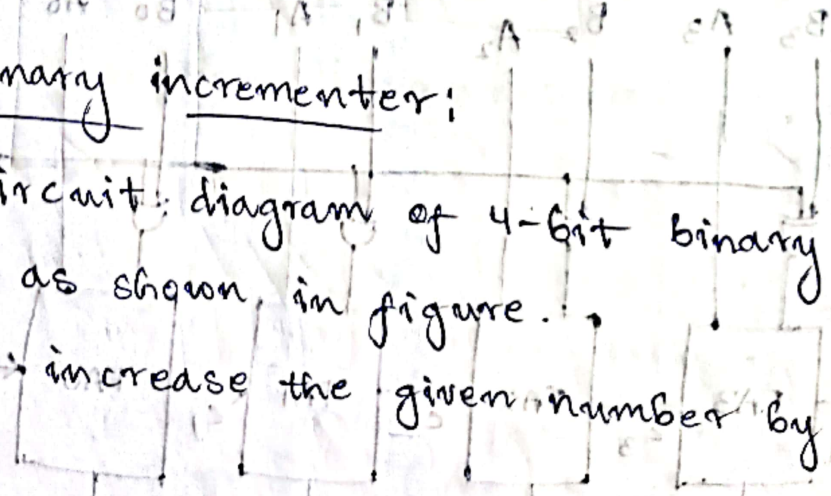
→ When M=0, addition is performed, in this case XOR pass the bits of B and the initial carry C_0 becomes '0' (zero). The full adders simply add A and B, that is A+B. So the circuit works as a normal 4-bit adder and produces a 4-bit sum.

→ When M=1, subtraction is performed, in this case XOR of each bit of B is complimented and the initial carry C_0 becomes '1' (one). This effectively forms the 2's c of B and the full adders perform, [A + (2's c of B)] which is A−B. So the circuit works as performs subtraction.

4-bit binary incrementer:

→ The circuit diagram of 4-bit binary incrementer as shown in figure.

→ It can increase the given number by 1.



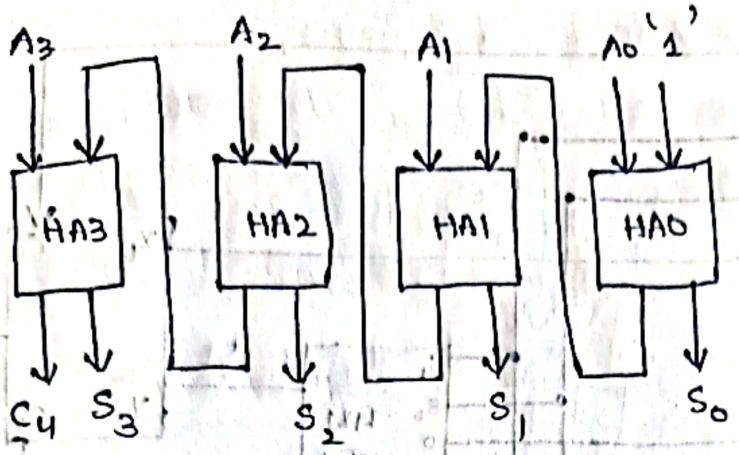


Fig: 4-bit binary incrementer.

→ The circuit uses four half adders. At LSB the HA receives A_0 and a constant input '1' and add together and produces sum bit S_0 and carry.

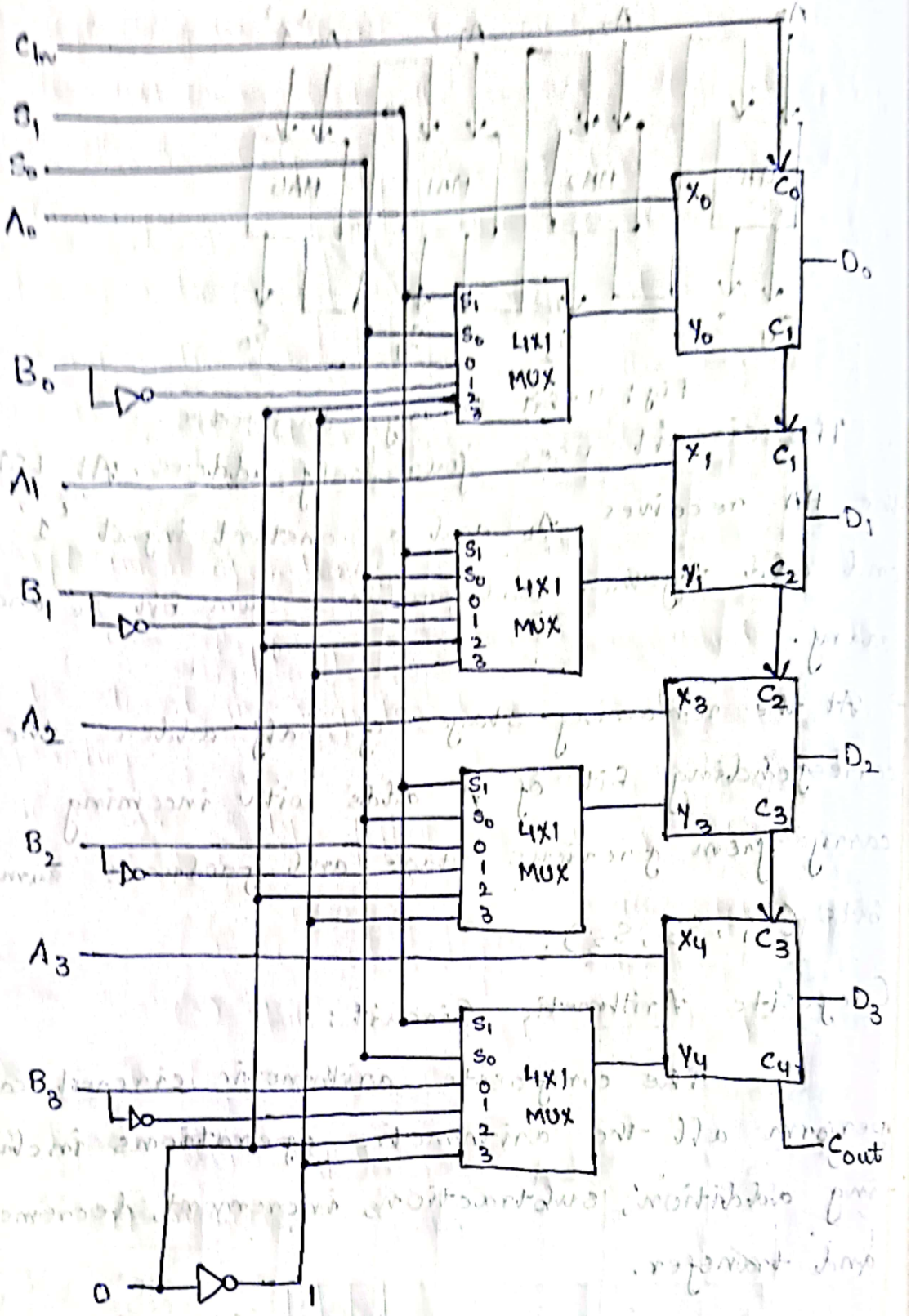
→ At the remaining stages of half adders the corresponding bit of A adds with incoming carry from previous stage and produces sum bits (S_1, S_2, S_3).

Composite Arithmetic Circuit:

The composite arithmetic circuit can perform all the arithmetic operations including addition, subtraction, increment, decrement and transfer.

→ The circuit diagram of composite arithmetic circuit as shown in figure below:

Circuit Diagram:



Select lines			Input Y (FA)	Output D = A+Y+Cin	Micro-operation
S_1	S_0	C_{in}			
0	0	0	B	D = A+B	Add
0	0	1	B	A+B+1	Add with carry
0	1	0	$\bar{B}$	$A+\bar{B}$	Sub with borrow
0	1	1	$\bar{B}$	$A+\bar{B}+1$	Subtract
1	0	0	0	A	Transfer A
1	0	1	0	A+1	Increment A
1	1	0	1	A−1	Decrement A
1	1	1	1	A	Transfer A

Scanned with OKEN Scanner

-> It consists of four 4x1 multiplexers with common selection lines S_1 and S_0 to select required arithmetic operation.

-> It also consists of four full adders to add or subtract if operands A and B

-> The functional discription of this arithmetic circuit showing fu shown in function table in which three selection lines S_1, S_0 and C_{in} selects any required arithmetic operation out of eight

Logical Micro-Operations:

The logical operations executed on the data stored in the registers are called logical micro-operations.

-> The important logical micro-operations are AND, OR, XOR and Complement.

-> The circuit diagram of one-bit 1-bit logical circuit shown in the figure below.

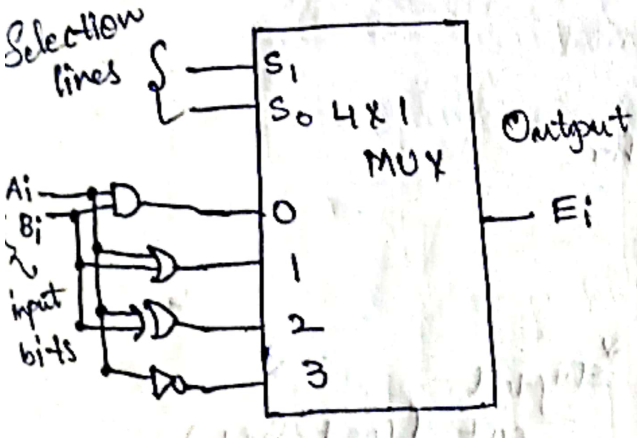


Fig. 1-bit logical circuit

Function Table:

Selection lines		Output (E_i)	Description
S_1	S_0		
0	0	$A_i \wedge B_i$	AND
0	1	$A_i \vee B_i$	OR
1	0	$A_i \oplus B_i$	XOR
1	1	$\bar{A}_i$	Complement

Scanned with OKEN Scanner

→ The circuit designed with a [illegible] the logical gates AND, OR, XOR, NOT.

→ The 4X1 MUX selects the required logical operation, with the help of selection lines (S_1 & S_0).

→ The logic gates perform the corresponding logic operations on inputs (A_i and B_i).

→ The functional description of the logical operation circuit is summarized in functional table.

Shift micro-operations:

The shifting operations executed on the data stored in registered are called shift micro-operations.

→ There are 8 types of shifting operations:

(1) Logical shift
(2) Circular shift
(3) Arithmetic shift

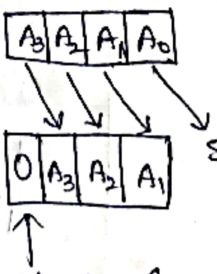
* The shift may be either left shift or right shift

(1) Logical shifting operations:

(1) Logical shift right:

notation → Shr

Shr A →



shifted out (lost bit)

Data in (I_R)

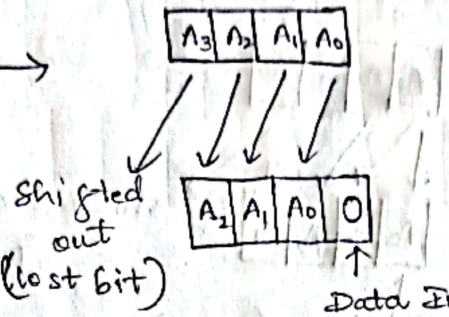
* In logic shift right, the data will be shifted by one position towards right. The LSB is shifted out

and lost, then the MSB is filled with '0'.

(ii) Logical shift left:

notation → shl

shl A →



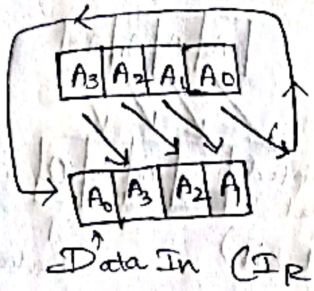
* In logic shift left, the data will be shifted by one position towards left. The MSB is shifted out and lost, then the LSB is filled with '0'.

(B) Circular shift:

(i) Circular shift right:

Notation → Cir

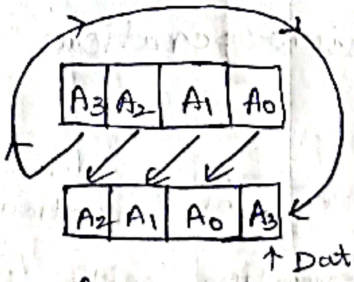
Cir A →



* In circular shift right, the data will be shifted by one position towards right, then the vacant MSB is filled with LSB (shifted one bit).

(ii) Circular shift left: Notation → Cil

Cil A →



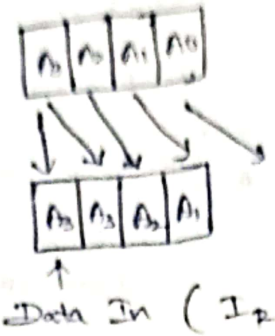
* In circular shift left, the data will be shifted by one position towards left, then the vacant LSB is filled with MSB

(3) Arithmetic shift:

(i) Arithmetic right shift:

Notation: ashr

ashr A:

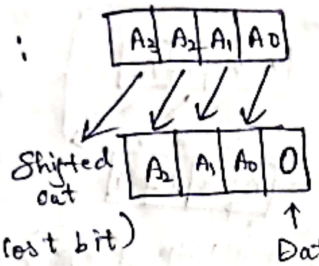


* In arithmetic right shift, the data will be shifted by one position, The LSB bit will be shifted out and lost, the vacant position filled with previous MSB.

(ii) Arithmetic shift left:

Notation: ashl

ashl A:



* In arithmetic shift left, the data will be shifted by one position towards left, the MSB will be shifted out and lost, then the LSB is filled with '0'.

Hardware Implementation:

The shifting micro-operations can be implemented in two ways.

1. By using bi-directional shift registers
2. implementing with flip flops.
2. By using combinational elements like multiplexers. These are called Combinational shifters.

Scanned with OKEN Scanner

4- bit combinational shifter:

→ The 4- bit combinational shifter can perform logical, circular, arithmetic shifting operations in either right shift or left shift.

→ The circuit diagram of 4-bit combinational shifter is shown below.

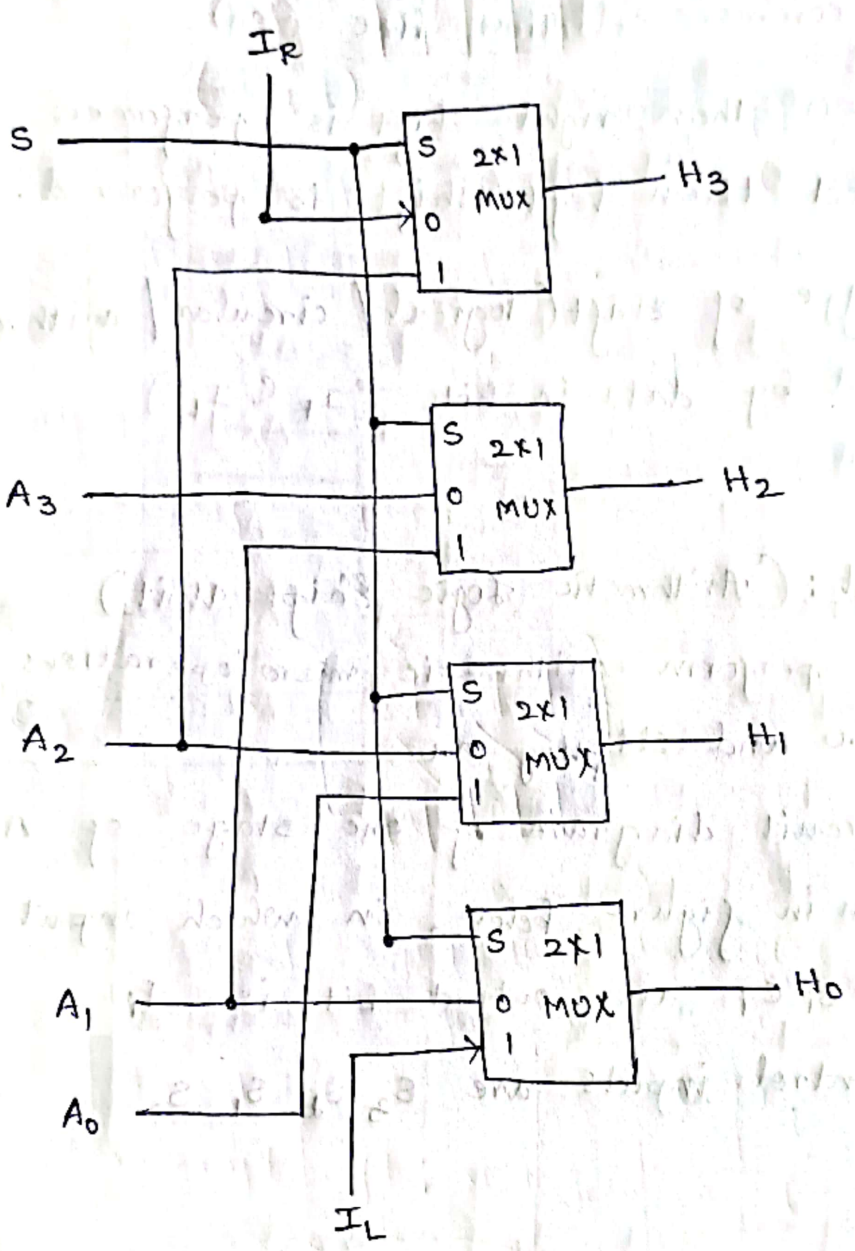


Fig: 4-bit combinational shifter.

Function Table: Input : A_3 A_2 A_1 A_0

Selection line (S)	Direction of shift	Output			
		H_3	H_2	H_1	H_0
0	Right	I_R	A_3	A_2	A_1
1	left	A_3	A_2	A_1	I_L

Scanned with OKEN Scanner

* for logical shift → $I_R = 0$ & $I_L = 0$
* for circular shift → $I_R = A_0$ & $I_L = A_3$
* for arithmetic shift → $I_R = A_3$ & $I_L = 0$

→ The circuit designed with four 2×1 MUXes, that select direction of shift (right or left) with the help of common selection line (S)

→ If S=0 then right shift is performed & If S=1 then left shift is performed.

→ The type of shift (logical / circular / arithmetic) is selected by data-in bits (I_R & I_L).

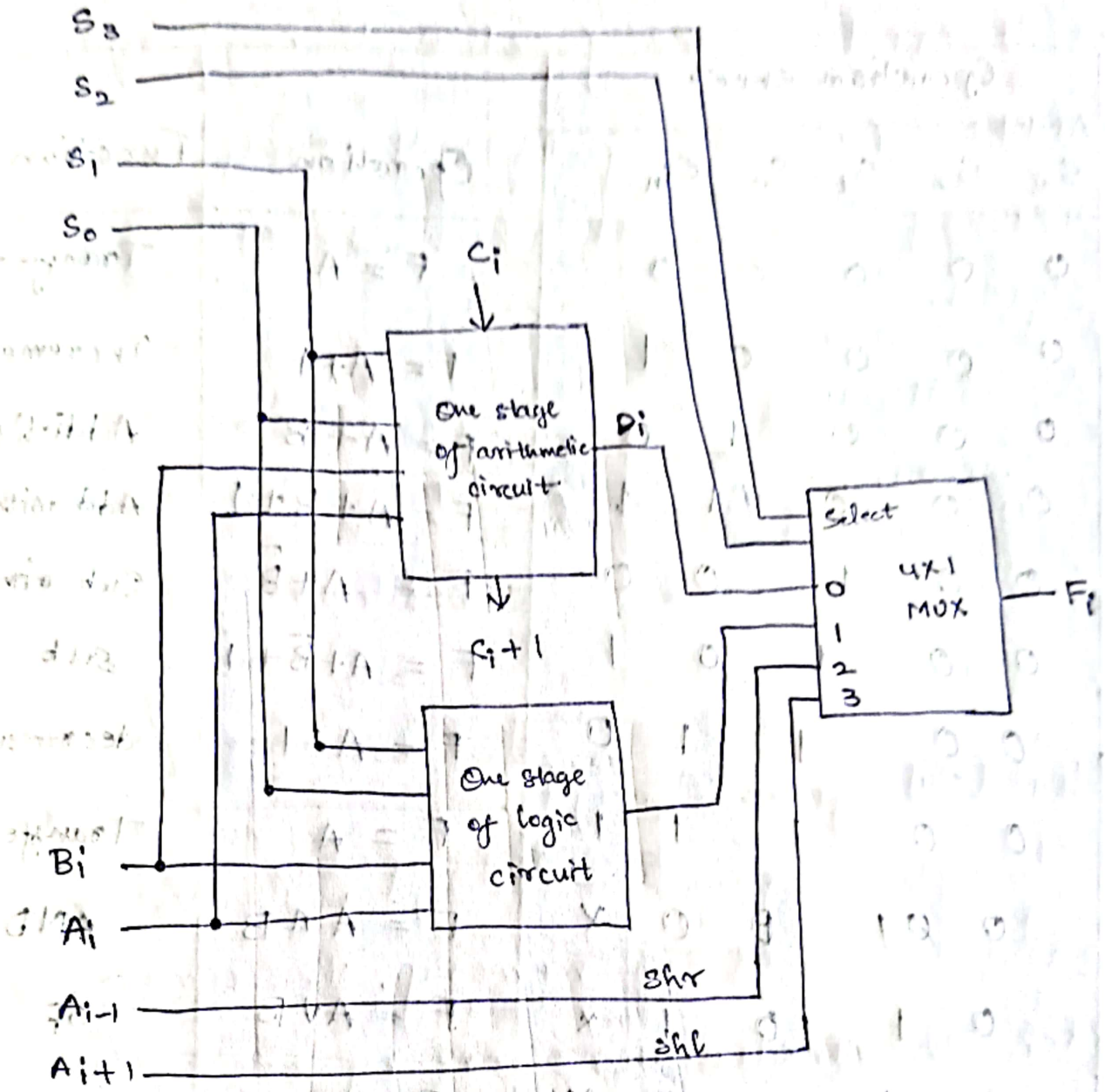
ALS unit: (Arithmetic Logic Shift Unit)

→ It can perform arithmetic micro operations, logical m.o and shifting m.o.

→ The circuit diagram of one stage of ALS unit as shown in figure below, in which input bits are A_i B_i C_i and output bit is F_i

→ The control inputs are S_3 S_2 S_1 S_0

Scanned with OKEN Scanner


S3
S2
S1
S0
Ci
One stage of arithmetic circuit
Di
Select
4x1 MUX
0
1
2
3
Fi
fi+1
One stage of logic circuit
Bi
Ai
Ai-1
Ai+1
shr
shl

Function table for arithmetic logic shift unit:

Operation sheet					Operation	Function
S_3	S_2	S_1	S_0	C_{in}		
0	0	0	0	0	F = A	Tranfer
0	0	0	0	1	F = A+1	Increment
0	0	0	1	0	F = A+B	Addition
0	0	0	1	1	F = A+B+1	Add with
0	0	1	0	0	F = A+$\bar{B}$	Sub with
0	0	1	0	1	F = A+$\bar{B}$+1	Sub
0	0	1	1	0	F = A-1	decrement
0	0	1	1	1	F = A	Tranfer
0	1	0	0	X	F = A ∧ B	AND
0	1	0	1	X	F = A ∨ B	OR
0	1	1	0	X	F = A ⊕ B	XOR
0	1	1	1	X	F = $\bar{A}$	complement
1	0	X	X	X	F = shr A	Shift right A into F
1	1	X	X	X	F = shl A	shift left A into F

→ In ALSU circuit, the arithmetic operations of one-stage provided by one stage arithmetic circuit block.

→ The logical operations are performed with the help of one stage of logic circuit block.

→ The shifting operations can be done directly.

Scanned with OKEN Scanner